MANIPAL INSTITUTE OF TECHNOLOGY

Multi-Strategy Quantitative Backtesting System

Portfolio Optimization & Risk Analysis

DBMS Mini Project Presentation

[Aditya Ranka / 240957140] || [Parth Bagal / 240957198] | CSS 2212 – Database Systems Lab

Project Overview

What is the System?

A full-stack platform simulating algorithmic trading strategies against historical data, computing risk metrics, and storing all simulation results in a relational database.

Strategies Supported

- Moving Average Crossover
- Momentum Strategy
- Mean Reversion
- RSI Reversal

Problem Statement

- Test strategies against historical data
- Calculate risk metrics: VaR, CVaR, Sharpe Ratio, Max Drawdown, Sortino Ratio
- Dynamically allocate capital across multiple strategies
- Store all simulation data, signals and results securely in a relational database

System Architecture & Database Design

Tech Stack

Frontend
React.js + Vite

Backend
Python 3 · Flask REST API

Database
SQLite (local) · PostgreSQL (prod)

Auth
JWT Tokens

Server
Apache / Nginx · Gunicorn

Core Database Tables

Users

user_id · username · email · created_at

Strategies

strategy_id · strategy_name · description

Strategy_Parameters

parameter_id · strategy_id · parameter_name · data_type

Market_Data

data_id · ticker_symbol · trade_date · OHLCV · adj_close

Backtest_Runs

run_id · user_id · strategy_id · ticker · start_date · end_date

Daily_Results

result_id · run_id · trade_date · signal · daily_return

Performance_Metrics

run_id · sharpe · sortino · cagr · max_drawdown · win_rate

Portfolio_Weights

weight_id · run_id · strategy_id · allocation_weight · date

Run_Parameter_Values

run_id · parameter_id · parameter_value

Data Integrity & Advanced Constraints

Foreign Keys

Referential Integrity

Every time-series record is bound to its parent run, strategy, and asset via cascading foreign keys. Deleting a backtest run automatically removes all associated results, metrics, and weights.

UNIQUE Constraints

No Duplicate Data

Market_Data enforces one OHLCV row per ticker per day. Daily_Results enforces one entry per run per date. Portfolio_Weights enforces one allocation per strategy per run per optimization date.

CHECK Constraints

Valid Values Only

Signal must be BUY, SELL, or HOLD. Win rate is bounded between 0 and 1. Allocation weight is bounded between 0 and 1. End date must be strictly greater than start date.

Cascade Deletes

ON DELETE CASCADE

All child records — Run_Parameter_Values, Daily_Results, Performance_Metrics, Portfolio_Weights — are automatically removed when their parent Backtest_Run is deleted.

Database Triggers for Business Logic

Trigger: trg_validate_portfolio_weights

Fires **AFTER INSERT OR UPDATE** on the **Portfolio_Weights** table to guarantee that the sum of all allocation weights for a given run and optimization date never exceeds 1.0 (100% of capital). The transaction is aborted if violated.

SQLite Implementation

Uses RAISE(ABORT) inside the trigger body to immediately abort the transaction when the summed weights exceed 1.0. Fires on INSERT.

PostgreSQL Implementation

Uses a PL/pgSQL RETURNS TRIGGER function with RAISE EXCEPTION. Covers both INSERT and UPDATE, ensuring the constraint is unconditional.

Why This Matters —

Integrity is enforced at the database layer, not just the application layer. Any client or service writing to Portfolio_Weights is subject to the same rule — no bypass is possible.

Stored Procedures & Views

VIEW — v_portfolio_allocation_history

Joins Portfolio_Weights with Strategies and Backtest_Runs to produce a clean, ordered history of strategy allocations per run and date.

Benefit: Eliminates complex nested joins in the application layer. Flask queries this view directly for portfolio tracking dashboards, keeping application code simple and maintainable.

STORED PROCEDURE — sp_get_run_summary

Accepts a run_id and returns ticker symbol, date range, Sharpe Ratio, Max Drawdown, and Total Return — computed via a server-side JOIN between Backtest_Runs and Performance_Metrics.

Benefit: Aggregation executes natively on the PostgreSQL server, offloading heavy computation from the Python backend and reducing API response time.

Conclusion

- Normalised 9-table relational schema ensures consistency and referential integrity across all entities
- CHECK, UNIQUE, and FOREIGN KEY constraints prevent invalid data at the database layer
- Trigger-based portfolio weight validation enforces business rules unconditionally
- Views and stored procedures offload complex queries from the application to the database server
- Composite indexes on time-series data deliver sub-second query performance

Future Work: AI-based return prediction • React Native mobile app • Zerodha or alternate platform integration